Final Project Report 1

Part 1 Data Preparation

1. As indicated in the pdf, we begin by preprocessing the documents. From the text fields we remove the stop words, tokenize, remove punctuation marks, and stem.

```python
import nltk
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download English stop words
nltk.download("stopwords")

# Initialize tools
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()

# Define preprocessing function
def preprocess_text(text):

    # 1. Convert to lowercase
    text = str(text).lower()

    # 2. Tokenization and punctuation removal
    tokens = re.findall(r"[a-z0-9]+", text)

    # 3. Remove stop words
    tokens = [word for word in tokens
              if word not in stop_words]

    # 4. Apply stemming
    tokens = [stemmer.stem(word) for word in tokens]

    return tokens

# Apply preprocessing to titles and descriptions
df["title_tokens"] = df["title"].apply(preprocess_text)
df["description_tokens"] = df["description"].apply(preprocess_text)

# Compare original and processed titles
print("Original title:")
print(df["title"].iloc[0])

print("\nProcessed title:")
print(df["title_tokens"].iloc[0])

print("\nProcessed description:")
print(df["description_tokens"].iloc[0])
```

2. With the following code we check that the required columns are still in the dataset:

```python
df_raw = df.copy()

# Required output schema specified in Part 1
output_fields = [
    "pid",
    "title",
    "description",
    "brand",
    "category",
    "sub_category",
    "product_details",
    "seller",
    "out_of_stock",
    "selling_price",
    "discount",
    "actual_price",
    "average_rating",
    "url"
]

# Check that every required field exists
missing_fields = [
    field for field in output_fields
    if field not in df_raw.columns
]

if missing_fields:
    raise ValueError(f"Missing required output fields: {missing_fields}")

# Select the fields in the required order
output_df = df_raw[output_fields].copy()

# Validate the output
assert output_df.columns.tolist() == output_fields
assert output_df["pid"].equals(df_raw["pid"])

print("Number of output fields:", len(output_df.columns))
print("Number of product records:", len(output_df))
print("Unique product IDs:", output_df["pid"].nunique())

display(output_df.head(3))
```

And in the output we clearly see them present:

pid	title	description	brand	category	sub_category	product_details	seller	out_of_stock	selling_price	discount	actual_price	average_rating	url
PFCZ9EA7H5FYZH	Solid Women Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	York	Clothing and Accessories	Bottomwear	[{'Style Code': '1005COMBO2'}, {'Closure': 'El...	Shyam Enterprises	False	921	69% off	2,999	3.9	https://www.flipkart.com/yorker-solid-men-mult...
PFCZ9EJZV2UVRZ	Solid Men Blue Track Pants	Yorker trackpants made from 100% rich combed c...	York	Clothing and Accessories	Bottomwear	[{'Style Code': '1005BLUE'}, {'Closure': 'Draw...	Shyam Enterprises	False	499	66% off	1,499	3.9	https://www.flipkart.com/yorker-solid-men-blue...
PFCZ9EHFCY5Z4Y	Solid Men Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	York	Clothing and Accessories	Bottomwear	[{'Style Code': '1005COMBO4'}, {'Closure': 'El...	Shyam Enterprises	False	931	68% off	2,999	3.9	https://www.flipkart.com/yorker-solid-men-mult...

3. Both category and subcategory are preprocessed as searchable text, but we keep them in separate fields so they can be weighed independently, since category describes a broad product group (like Clothing or Accessory) while subcategory describes a more specific group (Bottomwear, Shirt). For product_details what we do is to turn it into a searchable string. We flatten it (removing the original structure of the field) and then preprocess it. This approach has the limitation that, when we remove the original structure, we also lose the connection between different fields and values that the structure provided. Knowing this, we

don’t erase the original column but instead work on a derived one called product_details_text. Lastly, for brand and seller, we lowercase them and strip leading/trailing whitespace. However, we do not stem nor remove stop words because they are names or identifiers, and that’s information we aren’t willing to lose. In this approach we index all the fields independently. A different approach would be to merge all the fields into one, which could be simpler to implement, however this loses field identity and makes it harder to tune each field separately.
The code that performs these operations is the following:

```
# Convert product_details into readable text
# preprocessing of the text fields
def flatten_details(details):
    if isinstance(details, list):
        return " ".join(
            f"{key} {value}"
            for item in details if isinstance(item, dict)
            for key, value in item.items()
        )
    return str(details)

df["product_details_text"] = df["product_details"].apply(flatten_details)

# Preprocess categories and product details
for field in ["category", "sub_category", "product_details_text"]:
    df[field + "_tokens"] = df[field].apply(preprocess_text)

# Normalize brands and sellers without stemming their names
for field in ["brand", "seller"]:
    df[field + "_normalized"] = df[field].astype(str).str.strip().str.lower()
    df[field + "_tokens"] = df[field].apply(
        lambda x: re.findall(r"[a-z0-9]+", str(x).lower())
    )
```

4. For the numerical fields, we preserve these categories as structured values, not ordinary text (meaning if someone types 500 it’s being interpreted as a number, not text). We convert prices (selling_price, actual_price) and the ratings into numbers, removing spaces and non-numeric characters (commas, symbols), we convert the result into a numeric data type and store it in a new column of name [“original name” + num] (e.g. actual_price_num). Discounts are originally stored in a string format as “69% off”, so we extract the number from this expression and store it as a number in column discount_pct. Finally, out_of_stock is turned into a boolean variable. We use errors=coerce, which turns the values that couldn’t be converted into numbers into NaNs. After checking, all the missing values the code talks about were empty strings originally, so we are not missing any new information that was previously available on the dataset.
This is the code that processes numeric variables:

```python
# Convert price and rating fields into numbers
numeric_fields = ["selling_price", "actual_price", "average_rating"]

for field in numeric_fields:
    df[field + "_num"] = pd.to_numeric(
        df[field].astype(str)
        .str.replace(",", "", regex=False)
        .str.replace(r"[^\d.]", "", regex=True),
        errors="coerce"
    )

# Convert discount from "69% off" to 69
df["discount_pct"] = pd.to_numeric(
    df["discount"].astype(str)
    .str.extract(r"(\d+(?:\.\d+)?)")[0],
    errors="coerce"
)

# Keep stock availability as a Boolean
df["out_of_stock_bool"] = df["out_of_stock"].map({
    True: True,
    False: False
})

# Display first product
print(df[[
    "selling_price_num",
    "actual_price_num",
    "average_rating_num",
    "discount_pct",
    "out_of_stock_bool"
]].head())
```

It’s output including the description of missing values:

```
   selling_price_num  actual_price_num  average_rating_num  discount_pct  \
0              921.0            2999.0                 3.9          69.0
1              499.0            1499.0                 3.9          66.0
2              931.0            2999.0                 3.9          68.0
3              911.0            2999.0                 3.9          69.0
4              943.0            2999.0                 3.9          68.0

   out_of_stock_bool
0              False
1              False
2              False
3              False
4              False

Missing values after conversion:
selling_price_num        2
actual_price_num       777
average_rating_num    2261
discount_pct           855
out_of_stock_bool        0
dtype: int64
```

The code and output that explains these missing values:

```python
original_fields = {
    "selling_price": "selling_price_num",
    "actual_price": "actual_price_num",
    "average_rating": "average_rating_num",
    "discount": "discount_pct",
    "out_of_stock": "out_of_stock_bool"
}

for original, converted in original_fields.items():
    print(f"\n--- {original} ---")
    print("Missing in original:", df[original].isna().sum())
    print("Missing after conversion:", df[converted].isna().sum())

    # Show examples where a value existed originally but conversion failed
    failed = df[df[original].notna() & df[converted].isna()]
    print("Non-missing original values that failed:", len(failed))
    print(failed[original].head(10).tolist())
```

```
--- selling_price ---
Missing in original: 0
Missing after conversion: 2
Non-missing original values that failed: 2
['', '']

--- actual_price ---
Missing in original: 0
Missing after conversion: 777
Non-missing original values that failed: 777
['', '', '', '', '', '', '', '', '', '']

--- average_rating ---
Missing in original: 0
Missing after conversion: 2261
Non-missing original values that failed: 2261
['', '', '', '', '', '', '', '', '', '']

--- discount ---
Missing in original: 0
Missing after conversion: 855
Non-missing original values that failed: 855
['', '', '', '', '', '', '', '', '', '']

--- out_of_stock ---
Missing in original: 0
Missing after conversion: 0
Non-missing original values that failed: 0
[]
```

Part 2 Exploratory Data Analysis

Firstly, we tackle the numerical variables.

```
columns = [
    "selling_price_num",
    "actual_price_num",
    "average_rating_num",
    "discount_pct"
]

print("Summary statistics:")
print(df[columns].describe().round(2))
```

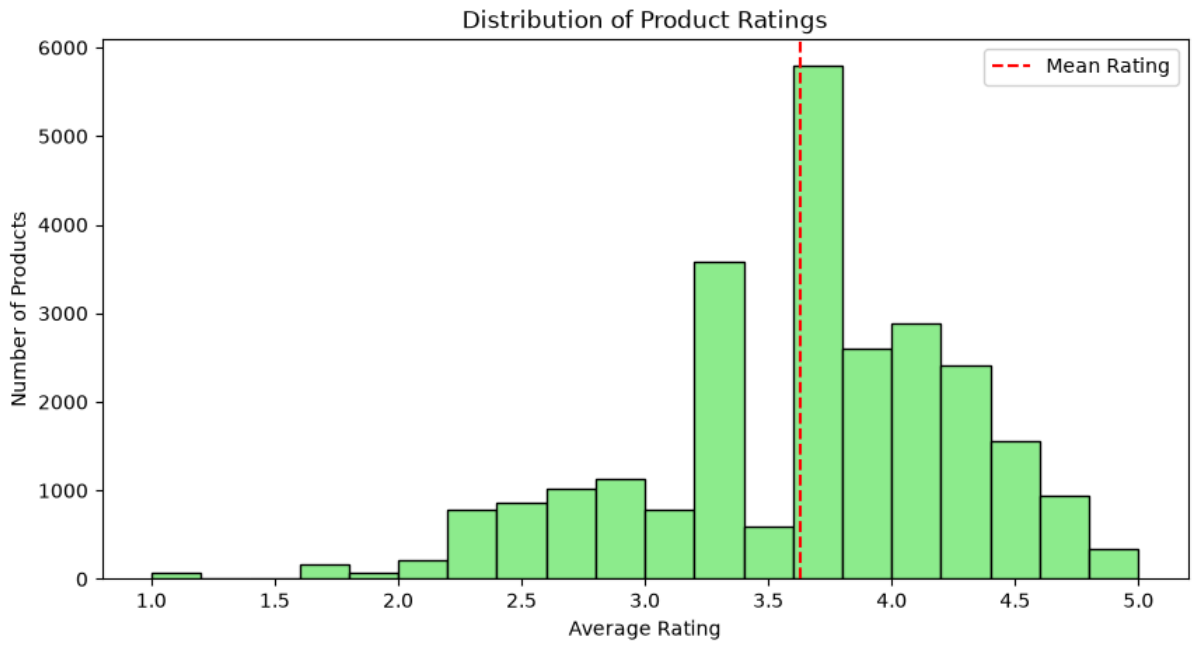
Summary statistics:

	selling_price_num	actual_price_num	average_rating_num	discount_pct
count	28078.00	27303.00	25819.00	27225.00
mean	705.64	1455.53	3.63	50.26
std	549.68	939.98	0.66	16.89
min	99.00	150.00	1.00	1.00
25%	390.00	849.00	3.20	40.00
50%	545.00	1199.00	3.80	53.00
75%	820.00	1799.00	4.10	63.00
max	7999.00	12999.00	5.00	87.00

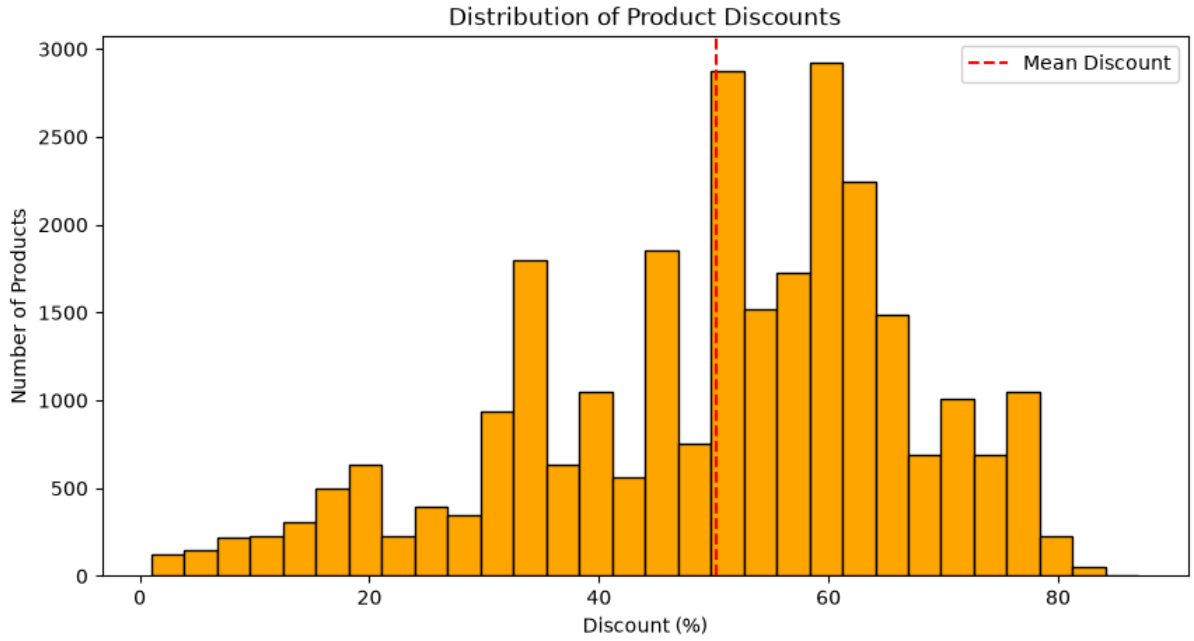
From these basic statistics we can see the mean and the median, allowing us to see, for example, that the distributions of selling_price_num, actual_price_num and discount_pct are right-skewed, since the mean is bigger than the median (represented by the 50% row). It is important to note that the missing values found earlier are not taken into account for these calculations (hence the different count per variable).



As we see from the distribution of the selling prices our above conclusions are right. Most of the prices fall under an approximate value of 500, but a small amount of more expensive products increases the mean (leading to the aforementioned right-skewed distribution).

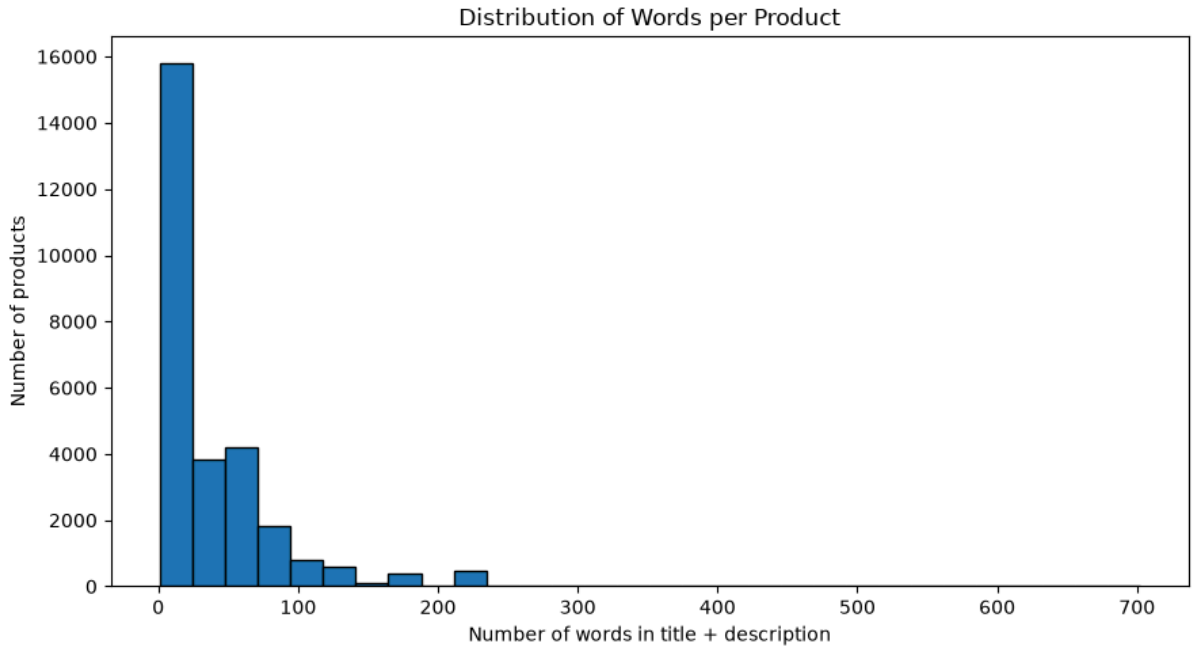


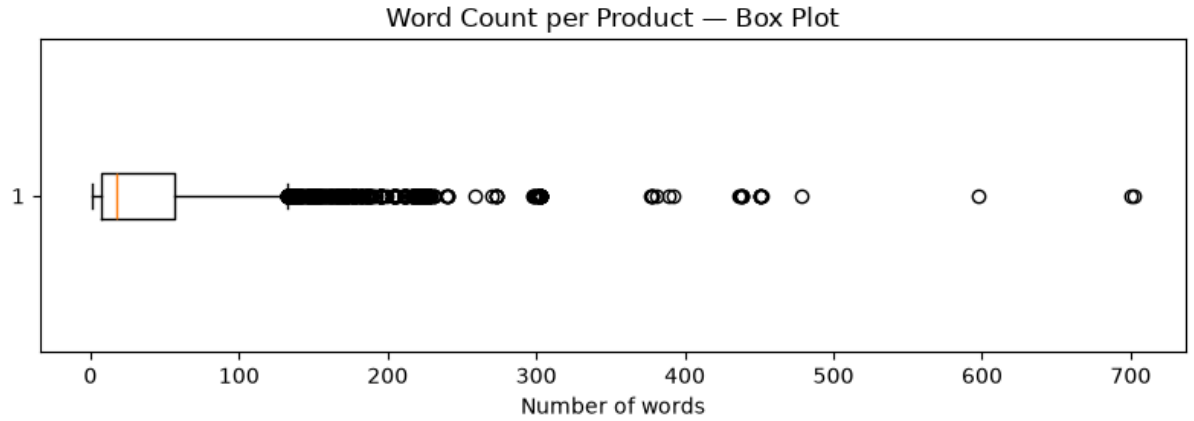
The ratings' histogram shows that most products have ratings between approximately 3.2 and 4.5, with an average rating of 3.63. Now we can notice that very low ratings are uncommon, which suggests that the majority of rated products receive relatively positive evaluations.



From the discount histogram we can see that there is an ample range of percentages from 5% up to more than 80%. Most of the discounts are found in the range of 50-60% with an average of 50.26%.

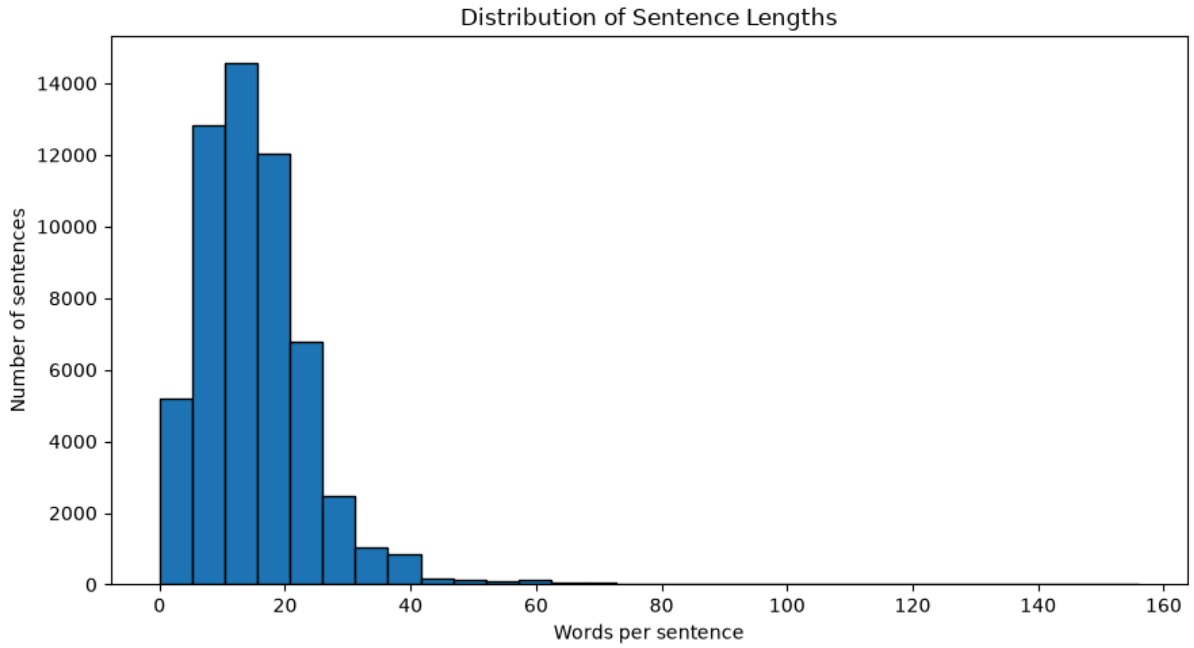
For some statistics about other variables, let's analyse the word count per product:



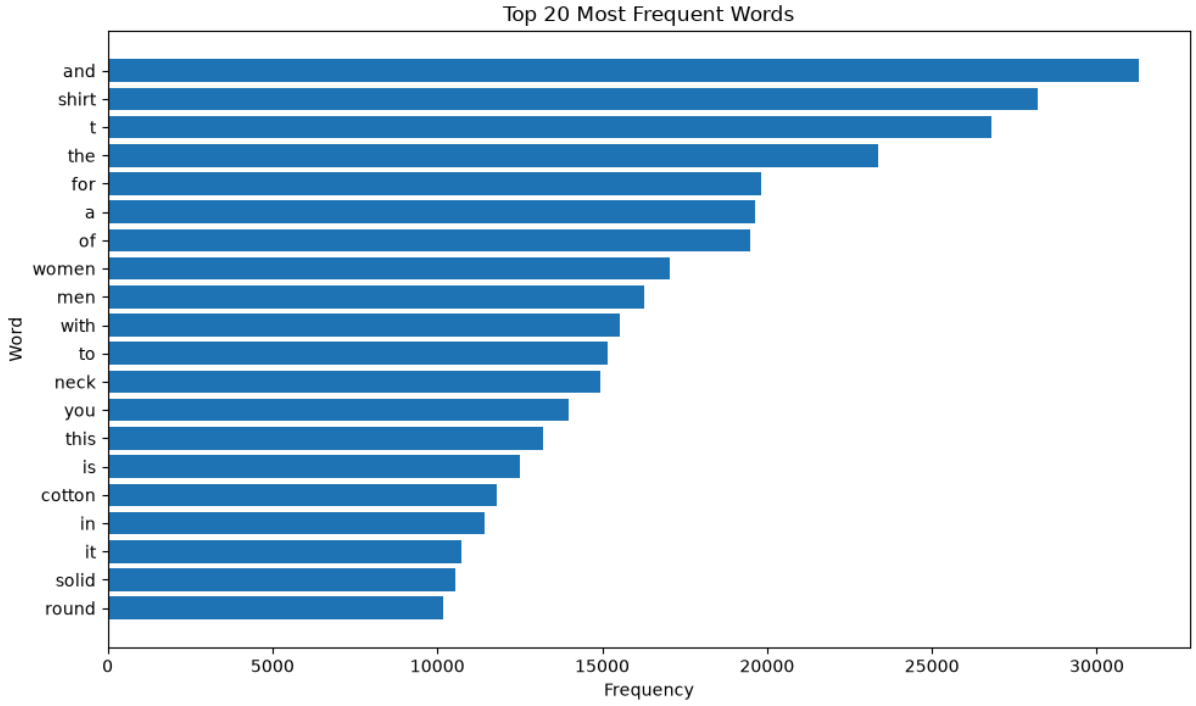


Here we can appreciate that most products have very little word counts, from this histogram we could say that most are less than 25 words, however, from the boxplot we could go even further and say that 50% of the products have 10 words or less. The boxplot also reveals some incredible outliers such as a few products with more than 700 words.

Similarly we can find the distribution of words per sentence, out of 56512 sentences found, where we find an average length of 15.24



Lastly, we can see the most frequent words. From the graph we can see many stop words, which are expected to be found in most sentences. However, we want to mention a surprising appearance in this top which is “t” as the third most frequent. This is better understood knowing that the second one is shirt, since for every ”T-shirt” this words appear.



AI declaration

AI tools were used for assistance in coding and debugging and in order to better comprehend the task ahead.